Nom i cognoms: …… SOLUCIO…… Signatura:…… ……

NOTA: NO es poden usar llistes, només arrays bàsics a Java. No poden haver-hi break dins d'iteratives, utilitzar els esquemes iteratius usats en classe. Comenta el codi. Controlar excepcions i caracteritzar les seqüències només en els apartats on se sol·liciti.

Exercici 1 (4 punts) La seqüència de "recompte i diu" és una seqüència d'enters, amb els següents primers sis termes:

1. 1
2. 11
3. 21
4. 1211
5. 111221
6. 312211

etc.

El primer terme és **1**, i com es llegeix **"un 1"** el segon terme és **11**
Com el segon terme és 11, i com es llegeix **"dos 1s"** el tercer terme és **21**
Com el tercer terme és 21, i com es llegeix **"un 2 i un 1"** el quart terme és **1211**
i així successivament…

Donat un enter n, heu de generar el n-èsim terme de la seqüència de recompte i diu. Cal tenir present que la generació d'aquest terme requereix prèviament la generació de tots els termes precedents. Cada terme de la seqüència es representarà com una cadena de caràcters. Per a n <= 0 s'ha de retornar la cadena "terme buit".

NOTA: Utilitza la classe `StringBuffer`, recorda que el mètode `append()` d'aquesta classe s'utilitza per a concatenar cadenes. A més, pots realitzar crides successives, per exemple:

```
StringBuffer cad = new StringBuffer("Hola");
cad.append(" Adeu").append(" Fins després"); //cad conté "Hola Adeu Fins després"
```

A) Escriu els mètodes `recompteIDiu()` i `obtenirSeguentSequencia()`. Llegir el javadoc (comentaris) abans de cada mètode per saber què fa cadascun d'ells, què reben i què retornen. El mètode `recompteIDiu()` crida a `obtenirSeguentSequencia()` **(3 punts)**

L'execució del `main()` de sota dona com sortida:

```
n: 0, Sortida: terme buit
n: 4, Sortida: 1211
```

```
public static void main(String[] args) {
    int n = 0;
    System.out.println("n: " + n + ", Sortida: " + recompteIDiu(n));
    n=4;
    System.out.println("n: " + n + ", Sortida: " + recompteIDiu(n));
}
/* Retorna el String que correspon al terme n-èsim de la seqüència recompte i diu
 * @param n nombre de terme de la seqüència
 * @return string amb el term enèsim de la seqüència recompte i diu
 */
public static String recompteIDiu(int n) {


      String result = "";

        if (n > 0) {
           result = "1";
            /*
             Identificació seq: n-1 termes de la seqüència
             Primer: 1
             Següent(i): i++
             FiSeq(i):  i >= n
             Esquema(): recorregut
           */

          for(int i = 1; i < n; ++i)
              result = obtenirSeguentSequencia(result);
      } else
          result = "terme buit";

      return result;
```

```
}
```

```
/* Genera el següent terme de la seqüència "Recompte i Diu" a partir d'una cadena donada.
 * El mètode analitza la cadena d'entrada, comptant el nombre de repeticions consecutives
 * de cada caràcter i genera una nova cadena que descriu el nombre de repeticions
 * seguit pel caràcter corresponent.
 * Per exemple, si la cadena d'entrada és "1211", el mètode generarà "111221" perquè:
 * - Hi ha un "1", un "2", i dos "1"s, és a dir: 111221
 * @param s La cadena d'entrada que representa el terme actual de la seqüència "Recompte i Diu".
 * @return Una cadena que representa el següent terme de la seqüència.
 *         Per exemple, si l'entrada és "1211", la sortida serà "111221".
 */
static String obtenirSeguentSequencia(String s) {


    StringBuffer seguentSequencia = new StringBuffer();
    int cont = 1;//comptador de nombre iguals a la cadena s
    // Iterem sobre la cadena per generar la seguent sequencia
    /*
       Identificació seq: cadascun dels caracters de la cadena (indexos dels cters)
       Primer: 1
       Següent(i): i++
       FiSeq(i):  i >= s.length
       Esquema(): recorregut
     */

    for(int i = 1; i < s.length(); ++i) {
        if (s.charAt(i) == s.charAt(i - 1))
            ++cont;
        else { //concatenem contador de nombres amb el caracter i inicio comptador
            seguentSequencia.append(cont).append(s.charAt(i - 1));
            cont = 1;
        }
    }
    //tractament del darrer element
    seguentSequencia.append(cont).append(s.charAt(s.length() - 1));
    return seguentSequencia.toString();


}
```

B) Utiliza JUnit per a testejar el mètode `recompteIDiu()`. Crea un mètode per a cada cas de test. **(0,5 punts)**

```
class Test {
    @Test
public void testRecompteIDiu_Entrada1() {
    int n = 1;
    String resultatEsperat = "1";
    String resultatObtingut = Recompte_I_Diu.recompteIDiu(n);
    assertEquals(resultatEsperat, resultatObtingut);
}

@Test
public void testRecompteIDiu_Entrada4() {
    int n = 4;
    String resultatEsperat = "1211";
    String resultatObtingut = Recompte_I_Diu.recompteIDiu(n);
    assertEquals(resultatEsperat, resultatObtingut);
}
@Test
public void testRecompteIDiu_Entrada0() {
    int n = 0;
    String resultatEsperat = "terme buit";
    String resultatObtingut = Recompte_I_Diu.recompteIDiu(n);
    assertEquals(resultatEsperat, resultatObtingut);
}
```

```java
@Test
public void testRecompteIDiu_EntradaNegativa() {
    int n = -3;
    String resultatEsperat = "terme buit";
    String resultatObtingut = Recompte_I_Diu.recompteIDiu(n);
    assertEquals(resultatEsperat, resultatObtingut);
}
```

} C) Abans de cridar a `recompteIDiu()` has de llegir per teclat el nombre `n`. Escriu de nou el `main()` amb un bloc `try-catch` que gestioni l'excepció que es genera quan l'entrada de dades és incorrecta. Si l'usuari introdueix qualsevol cosa que no sigui un nombre enter (caràcter, string,…), guarda en `n` el valor 0. **(0,5 punts)**

```java
public static void main(String[] args) {



    Scanner sc = new Scanner(System.in);
    try {
        System.out.println("Introdueix n: ");
        n = sc.nextInt();
    }
    catch(InputMismatchException i){
        n=0;
    }
    finally{
        sc.close();
    }


   …continua aqui main()…



}
```

Exercici 2 (4 punts) Donat un array `n x n` que representa una imatge, gira la imatge 90 graus (en el sentit horari de les agulles del rellotje). Per girar l'imatge, primer s'ha de trasposar la matriu i després, sobre la matriu trasposada, s'han d'invertir les files. IMPORTANT: Totes les operacions es fan sobre el mateix array, sense utilitzar un array auxiliar.

L'execució del `main()` a sota dona com sortida:

Array 1 abans de rotar:
1 0 0
1 0 0
1 1 1

Array 1 després de rotar:
1 1 1
1 0 0
1 0 0

Array 2 abans de rotar:
 1 2 3 4
 5 6 7 8
 9 10 11 12
13 14 15 16

Array 2 després de rotar:
13 9 5 1
14 10 6 2

15 11 7 3
16 12 8 4

A) Analitza el `main()` que es mostra a continuació i implementa els mètodes `trasposar()` i `invertirFiles()`. Llegir els comentaris abans de cada mètode per saber què fa cadascun d'ells. **(1,5 punts cada mètode)**

```
public class RotarArray {
    public static void main(String[] args) {
        int[][] array1 = {
          {1, 0, 0},
          {1, 0, 0},
          {1, 1, 1}};
        int[][] array2 = {
          {1, 2, 3, 4},
          {5, 6, 7, 8},
          {9, 10, 11, 12},
          {13, 14, 15, 16}};
        };

        System.out.println("Array 1 abans de rotar:");
        mostrar(array1);
        rotar(array1);
        System.out.println("Array 1 després de rotar:");
        mostrar(array1);

        System.out.println("Array 2 abans de rotar:");
        mostrar(array2);
        System.out.println("Array 2 després de rotar:");
        rotar(array2);
        mostrar(array2);
    }
    static void rotar(int[][] array) {
        trasposar (array);
        invertirFiles(array);
    }

    /**
     * Realitza la transposició d'una matriu quadrada d'enters (n x n).
     * En una transposició de matriu, els elements de la matriu es intercanvien
     * de manera que la fila i es converteix en la columna i
     * per a tots els índexs i, j. És a dir,
     * l'element a la posició (i, j) es trasllada a la posició (j, i).
     *
     * Per exemple, si la matriu d'entrada és:
     * 1 2 3
     * 4 5 6
     * 7 8 9
     * Després cridar aquest mètode, la matriu serà:
     * 1 4 7
     * 2 5 8
     * 3 6 9
     *
     * @param array La matriu quadrada d'enters (n x n) que es vol transposar.
     *              La matriu es modifica directament, no es retorna cap valor.
     */
    static void trasposar(int[][] array) {

      int n = array.length;
      for (int i = 0; i < n; i++) {

          //intercamvi elements en [0][1] amb [1][0], [0][2] amb [2][0] etc.
          //diagonals iguals, no les toco
          for (int j = i + 1; j < n; j++) {
              int temp = array[i][j];
              array[i][j] = array[j][i];
              array[j][i] = temp;
          }
      }
```

```
}

    /**
     * Inverteix les files d'una matriu quadrada de dimensions n x n.
     * El mètode intercanvia els elements de cada fila de manera que la fila s'inverteixi, és a dir,
     * els elements de la primera columna es col·loquen a l'última columna,
     * els de la segona a l'abans de l'última, i així successivament.
     *
     * Per exemple, si la matriu és:
     * 1  2  3
     * 4  5  6
     * 7  8  9
     * Després d'aplicar aquest mètode, la matriu serà:
     * 3  2  1
     * 6  5  4
     * 9  8  7
     *
     * @param array La matriu quadrada d'enters que es vol invertir fila per fila.
     *              La matriu s'ha de modificar directament, no es retorna cap valor.
     */

static void invertirFiles(int[][] array) {


      int n = array.length;

      for (int i = 0; i < n; i++) {

          for (int j = 0; j < n / 2; j++) {
              int temp = array[i][j];
              array[i][j] = array[i][n - 1 - j];
              array[i][n - 1 - j] = temp;
          }
      }

}



 static void mostrar(int[][] array2){
  for (int[] fila : array2) {
     for (int val : fila) {
       System.out.print(val + " ");
     }
     System.out.println();
  }
  System.out.println();
 }
}
```

B) El paràmetre `array` del mètode `invertirFiles()`, és un parámetre d'entrada, de sortida, o ambdós? Raona la teva resposta. **(0,5 punts)**

Es de entrada y sortida, d'entrada ja que el paràmetre ve amb dades i, de sortida perque dintre del mètode modifiquem l'array al ser un tipus referencia. En acabar el mètode l'array es veurà modificat des de el mètode on es crida (en aquest cas rotar()).

C) Si has utilitzat sentències iteratives a `invertirFiles()`, fes la caracterització de les seqüències. **(0,5 punts)**

```
/*
   Identificació seq: posiciones filas array
   Primer: 1
   Següent(i): i++
   FiSeq(i):  i >= n
   Esquema(): recorregut
 */
for (int i = 0; i < n; i++) {
    /*
   Identificació seq: posiciones columnas array
   Primer: 1
   Següent(j): j++
   FiSeq(j):  j >= n/2
   Esquema(): recorregut
 */
    for (int j = 0; j < n / 2; j++) {

....
```

Exercici 3. **(2 punts)** Dos cangurs avancen en el mateix sentit saltant, alhora, sobre una línia recta. Volem trobar si en algun punt coincidiran; per fer-ho tenim les dues posicions inicials dels cangurs (x1, x2) i les longituds dels seus salts (s1, s2).

La execució del `main()` a sota dona la següent sortida:

```
true
false
```

```
public class Cangurs {
  public static void main(String[] args) {

    System.out.println(coincideixen(2,1,1,2));
    System.out.println(coincideixen(2,1,2,1));
  }
  /* Metode per veure si els dos cangurs coincideixen
   * @param x1 posicio inicial del cangur 1    @param x2 posicio inicial del cangur 2
   * @param s1 longitud del salt del cangur 1  @param s2 longitud del salt del cangur 2
   * @return true si coincideixen en algun punt, false en cas contrari
   */

//Aquesta és una solució iterativa. A més, assumim que x1 pot ser igual a x2 (x1==x2)
   static boolean coincideixen(int x1, int x2, int s1, int s2){

       //Intercanviem per assegurar-nos que el cangur (x1, s1) es el que comenca mes a l'esquerra
       //si la seva posició (x1) es major que l'altre (x2), o si son iguals pero té
       // més salt (no volem entrar al else if d'abaix ja que no hi ha possibilitats de coincidir
       // quan salten)
       if (x1 > x2 || (x1 == x2 && s1 > s2)) {
           int aux = x2; x2 = x1; x1 = aux;
           int sAux = s2; s2 = s1; s1 = sAux;
       }

       //si mateix salt, coincideixen si mateixa pos inicial
       if (s1 == s2)
           coincideixen = x1 == x2;
       // si cangur mes a la esquerra te més salt que l'altre,
       //tenen possibilitats de coincidir
       else if (s1 > s2) {
           // mentre el cangur (x1, s1) es troba a l'esquerra
           //  els faig saltar
           while (x1 < x2) {
               x1 += s1;
               x2 += s2;
           }
           coincideixen = x1 == x2; //true coincideixen, false ha adelantat i marxa
```

```
        }
        return coincideixen;
    }

}
//Altre solució no iterativa, planteja una funcio lineal per calcular nombre de salts per a coincidir
//coincideixen en n salts quan: x1+s1*n=x2+s2*n, aïllo n
//n=(x2-x1)/(s1-s2)

static boolean coincideixen(int x1, int x2, int s1, int s2){
    boolean coincideixen = false;
    //comencen a la mateixa posició i mateix salt, al primer salt ja coincideixen
    if (x1==x2 && s1==s2)
        coincideixen = true;
    //comencen a la mateixa posició però amb diferent salt, mai coincidiran
    else if (x1==x2 && s1!=s2)
       coincideixen = false;
    else
       coincideixen = (s1!=s2 && (x1-x2)%(s2-s1)==0);

    return coincideixen;
}
```